NAME: M. Loganath

REG NO.:192424399

COURSE CODE: CSA1748

COURSE NAME: Artificial Intelligence For Healthcare Systems

1. Write a python program to implement Simple Calculator program?

```python
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    if y == 0:
        return "Error! Division by zero."
    return x / y3


print("Select operation:")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Divide")

choice = input("Enter choice (1/2/3/4): ")

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

if choice == '1':
    print("Result:", add(num1, num2))
elif choice == '2':
    print("Result:", subtract(num1, num2))
elif choice == '3':
    print("Result:", multiply(num1, num2))
elif choice == '4':
    print("Result:", divide(num1, num2))
else:
    print("Invalid input")
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/minmax.py ==
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4): 3
Enter first number: 5
Enter second number: 9
Result: 45.0
>>>
```

2. Write a python program to Add Two Matrices.

```python
# Program to add two matrices

# Define two matrices
A = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]

B = [[9, 8, 7],
     [6, 5, 4],
     [3, 2, 1]]

# Initialize result matrix with zeros
result = [[0 for _ in range(len(A[0]))] for _ in range(len(A))]

# Add matrices element by element
for i in range(len(A)):
    for j in range(len(A[0])):
        result[i][j] = A[i][j] + B[i][j]

# Print the result
print("Matrix A:")
for row in A:
    print(row)

print("\nMatrix B:")
for row in B:
    print(row)

print("\nSum of A and B:")
for row in result:
    print(row)
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/minmax.py ==
Matrix A:
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

Matrix B:
[9, 8, 7]
[6, 5, 4]
[3, 2, 1]

Sum of A and B:
[10, 10, 10]
[10, 10, 10]
[10, 10, 10]
>>>
```

3. Write a python program to Transpose a Matrix.

```python
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]
rows = len(matrix)
cols = len(matrix[0])
transpose = [[0 for _ in range(rows)] for _ in range(cols)]
for i in range(rows):
    for j in range(cols):
        transpose[j][i] = matrix[i][j]
print("Original Matrix:")
for row in matrix:
    print(row)
print("\nTransposed Matrix:")
for row in transpose:
    print(row)
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\matrix.py ==
Original Matrix:
[1, 2, 3]
[4, 5, 6]

Transposed Matrix:
[1, 4]
[2, 5]
[3, 6]
>>>
```

4. Write a python program to sort the sentence in alphabetical order?

```python
# Python program to sort the words in a sentence alphabetically

# Input sentence
sentence = "Python is a powerful and easy to learn programming language"

# Split sentence into words
words = sentence.split()

# Sort words alphabetically
words.sort()

# Display sorted words
print("Original Sentence:")
print(sentence)

print("\nSorted Words in Alphabetical Order:")
for word in words:
    print(word)
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/APPEND.py ==
Original Sentence:
Python is a powerful and easy to learn programming language

Sorted Words in Alphabetical Order:
Python
a
and
easy
is
language
learn
powerful
programming
to
```

5. Write a python program to implement List operations (Nested List, Length, Concatenation, Membership, Iteration, Indexing and Slicing)?

```python
# 1. Nested List
nested_list = [[1, 2, 3], [4, 5, 6], ['a', 'b', 'c']]
print("Nested List:", nested_list)
print("Access element '5' from nested list:", nested_list[1][1])
print("Access element 'c' from nested list:", nested_list[2][2])

# 2. Length of List
my_list = [10, 20, 30, 40, 50]
print("\nList:", my_list)
print("Length of List:", len(my_list))

# 3. Concatenation of Lists
list1 = [1, 2, 3]
list2 = [4, 5, 6]
concat_list = list1 + list2
print("\nConcatenation of list1 and list2:", concat_list)

# 4. Membership Test
print("\nIs 20 in my_list?", 20 in my_list)
print("Is 100 not in my_list?", 100 not in my_list)

# 5. Iteration through List
print("\nIterating through my_list:")
for item in my_list:
    print(item, end=" ")

# 6. Indexing
print("\n\nIndexing examples:")
print("First element:", my_list[0])
print("Last element:", my_list[-1])
print("Third element:", my_list[2])

# 7. Slicing
print("\nSlicing examples:")
print("Elements from index 1 to 3:", my_list[1:4])
print("Elements from start to index 2:", my_list[:3])
print("Elements from index 2 to end:", my_list[2:])
print("Every second element:", my_list[::2])
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/APPEND.py ==
Nested List: [[1, 2, 3], [4, 5, 6], ['a', 'b', 'c']]
Access element '5' from nested list: 5
Access element 'c' from nested list: c

List: [10, 20, 30, 40, 50]
Length of List: 5

Concatenation of list1 and list2: [1, 2, 3, 4, 5, 6]

Is 20 in my_list? True
Is 100 not in my_list? True

Iterating through my_list:
10 20 30 40 50

Indexing examples:
First element: 10
Last element: 50
Third element: 30

Slicing examples:
Elements from index 1 to 3: [20, 30, 40]
Elements from start to index 2: [10, 20, 30]
Elements from index 2 to end: [30, 40, 50]
Every second element: [10, 30, 50]
>>>
```

6. Write a python program to implement List methods (Add, Append, Extend & Delete).

```python
my_list = [10, 20, 30]
print("Initial List:", my_list)

my_list.append(40)
print("After Append (40):", my_list)

my_list.extend([50, 60, 70])
print("After Extend ([50, 60, 70]):", my_list)

my_list.insert(2, 25)  # Insert 25 at index 2
print("After Insert (25 at index 2):", my_list)

my_list.remove(30)  # Removes the first occurrence of 30
print("After Remove (30):", my_list)

popped_element = my_list.pop(3)  # Removes element at index 3
print(f"After Pop (index 3, removed {popped_element}):", my_list)

del my_list[0]
print("After del (index 0):", my_list)
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/APPEND.py ==
Initial List: [10, 20, 30]
After Append (40): [10, 20, 30, 40]
After Extend ([50, 60, 70]): [10, 20, 30, 40, 50, 60, 70]
After Insert (25 at index 2): [10, 20, 25, 30, 40, 50, 60, 70]
After Remove (30): [10, 20, 25, 40, 50, 60, 70]
After Pop (index 3, removed 40): [10, 20, 25, 50, 60, 70]
After del (index 0): [20, 25, 50, 60, 70]
>>>
```

7. Write a python program to Illustrate Different Set Operations?

```python
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

print("Set A:", A)
print("Set B:", B)

print("\nUnion of A and B:", A | B)

print("Intersection of A and B:", A & B)

print("Difference of A and B (A - B):", A - B)
print("Difference of B and A (B - A):", B - A)

print("Symmetric Difference of A and B:", A ^ B)

print("\nIs A subset of B?", A.issubset(B))
print("Is A superset of B?", A.issuperset(B))

print("Are A and B disjoint?", A.isdisjoint(B))
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/setoperation.py
Set A: {1, 2, 3, 4, 5}
Set B: {4, 5, 6, 7, 8}

Union of A and B: {1, 2, 3, 4, 5, 6, 7, 8}
Intersection of A and B: {4, 5}
Difference of A and B (A - B): {1, 2, 3}
Difference of B and A (B - A): {8, 6, 7}
Symmetric Difference of A and B: {1, 2, 3, 6, 7, 8}

Is A subset of B? False
Is A superset of B? False
Are A and B disjoint? False
>>>
```

8. Write a python program to generate Calendar for the given month and year?

matrix ai.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\matrix ai.py (3.13.7)

```python
import calendar

year = int(input("Enter a year: "))
month = int(input("Enter a month (1-12): "))

print(calendar.month(year, month))
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\matrix ai.py =
Enter a year: 2026
Enter a month (1-12): 6
     June 2026
Mo Tu We Th Fr Sa Su
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30
```

9. Write a python program to remove punctuations from the given string?

matrix ai.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\matrix ai.py (3.13.7)

```python
a = input("Enter string: ")
punctuations = '!'"()-[]{};:"\,<>./?@#$%^&*_~'
result = ""

for char in a:
    if char not in punctuations:
        result = result + char

print(result)
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
Warning (from warnings module):
  File "C:\Users\logan\AppData\Local\Programs\Python\Python313\matrix ai.py", line 2
    punctuations = '!'"()-[]{};:"\,<>./?@#$%^&*_~'
SyntaxWarning: invalid escape sequence '\,'
>>>
= RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\matrix ai.py =
Enter string: xgy,;n
xgyn
>>>
```

10. Write the python program to solve 8-Puzzle problem

8-Puzzle.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\8-Puzzle.py (3.13.7)

```python
    open_heap = []
    heappush(open_heap, (h_func(start, goal), 0, start))
    came_from = {start: None}
    g_score = {start: 0}

    while open_heap:
        f, g, current = heappop(open_heap)

        if current == goal:
            path = []
            s = current
            while s is not None:
                path.append(s)
                s = came_from[s]
            path.reverse()
            return path, g, None

        for nb in neighbors(current):
            tentative_g = g_score[current] + 1
            if nb not in g_score or tentative_g < g_score[nb]:
                g_score[nb] = tentative_g
                f_nb = tentative_g + h_func(nb, goal)
                came_from[nb] = current
                heappush(open_heap, (f_nb, tentative_g, nb))

    return None, None, "No solution found"

def print_board(state):
    for i in range(0, 9, 3):
        row = state[i:i+3]
        print(" ".join(str(x) if x != 0 else "_" for x in row))
    print()

if __name__ == "__main__":
    start = to_tuple([1, 2, 3,
                      4, 0, 6,
                      7, 5, 8])

    goal = to_tuple([1, 2, 3,
                     4, 5, 6,
                     7, 8, 0])

    path, cost, err = a_star(start, goal, heuristic="manhattan")
    if err:
        print("Error:", err)
    else:
        print(f"Solved in {cost} moves, steps below:")
        for step_idx, state in enumerate(path):
            print(f"Step {step_idx}:")
            print_board(state)
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\8-Puzzle.py =
Solved in 2 moves, steps below:
Step 0:
1 2 3
4 _ 6
7 5 8

Step 1:
1 2 3
4 5 6
7 _ 8

Step 2:
1 2 3
4 5 6
7 8 _
>>>
```

11. Write the python program to solve 8-Queen problem

8 queen.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\8 queen.py (3.13.7)

```python
def solve_queens(n):
    board = [-1] * n
    result = []

    def is_valid(row, col):
        for i in range(row):
            if board[i] == col or abs(row - i) == abs(col - board[i]):
                return False
        return True

    def backtrack(row):
        if row == n:
            result.append(list(board))
            return
        for col in range(n):
            if is_valid(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1

    backtrack(0)
    return result


# Solve for 8 queens
solutions = solve_queens(8)

# Print only the first solution
solution = solutions[0]   # pick the first one
print("Single Solution:\n")
for row in range(8):
    line = ""
    for col in range(8):
        if solution[row] == col:
            line += "♛ "   # queen
        else:
            # alternate chessboard squares
            line += "□ " if (row + col) % 2 == 0 else "■ "
    print(line)
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\8 queen.py ==
Single Solution:

...
♛ ■ □ ■ □ ■ □ ■
■ □ ■ □ ♛ □ ■ □
□ ■ □ ■ □ ■ □ ♛
■ □ ■ □ ■ ♛ ■ □
□ ■ ♛ ■ □ ■ □ ■
■ □ ■ □ ■ □ ♛ □
□ ♛ □ ■ □ ■ □ ■
■ □ ■ ♛ ■ □ ■ □
>>>
```

12. Write the python program for Water Jug Problem

water jug.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\water jug.py (3.13.7)

File Edit Format Run Options Window Help

```python
from collections import deque

def water_jug_problem(x, y, z):
    """
    Solves the water jug problem using BFS.
    x: capacity of jug 1
    y: capacity of jug 2
    z: target amount
    Returns the path of states to reach the target.
    """
    queue = deque([(0, 0, [])])  # (jug1, jug2, path)
    visited = set()

    while queue:
        a, b, path = queue.popleft()

        if (a, b) in visited:
            continue
        visited.add((a, b))

        # Check goal
        if a == z or b == z:
            path.append((a, b))
            return path

        # Generate next states (no action strings, just states)
        next_states = [
            (x, b),
            (a, y),
            (0, b),
            (a, 0),
            (a - min(a, y - b), b + min(a, y - b)),
            (a + min(x - a, b), b - min(x - a, b))
        ]

        for new_a, new_b in next_states:
            queue.append((new_a, new_b, path + [(a, b)]))

    return None


# Example run
path = water_jug_problem(4, 3, 2)

if path:
    print("Path of states by jugs followed is:")
    for state in path:
        print(f"{state[0]}, {state[1]}")
else:
    print("No solution found.")
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\water jug.py =
Path of states by jugs followed is:
0, 0
0, 3
3, 0
3, 3
4, 2
>>>
```

13. Write the python program for Cript-Arithmetic problem

Cript-Arithmetic Problem.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\Cript-Arithmetic Pr...

File Edit Format Run Options Window Help

```python
from itertools import permutations

puzzle = "SEND + MORE = MONEY"
addends = ["SEND", "MORE"]
result = "MONEY"

letters = list(set("".join(addends) + result))
leading = {w[0] for w in addends + [result]}

for perm in permutations('0123456789', len(letters)):
    mapping = dict(zip(letters, perm))
    if any(mapping[ch] == '0' for ch in leading):
        continue

    def val(word): return int("".join(mapping[c] for c in word))
    if sum(val(w) for w in addends) == val(result):
        print(f"{addends[0]} = {val(addends[0])}")
        print(f"{addends[1]} = {val(addends[1])}")
        print(f"{result} = {val(result)}")
        break
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\Cript-Arithmetic Probl
em.py
SEND = 9567
MORE = 1085
MONEY = 10652
>>>
```

14. Write the python program for Missionaries Cannibal problem

Missionaries Cannibal problem.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\Missionaries...

File Edit Format Run Options Window Help

```python
from typing import List, Tuple
from collections import deque

def is_valid_state(state: Tuple[int, int, int, int, int, int]) -> bool:
    """
    Check if a state is valid according to the problem constraints.
    State format: (m1, c1, boat, m2, c2, boat_side)
    """
    m1, c1, b, m2, c2, _ = state

    # No negative values allowed
    if m1 < 0 or c1 < 0 or m2 < 0 or c2 < 0:
        return False

    # No more cannibals than missionaries on either side (if missionaries exist)
    if (m1 > 0 and m1 < c1) or (m2 > 0 and m2 < c2):
        return False

    return True


def get_successors(state: Tuple[int, int, int, int, int, int]) -> List[Tuple[int, int, int, i
    """
    Generate all possible valid successor states.
    """
    m1, c1, b, m2, c2, d = state
    successors = []

    if d == 1:
        # Boat is on the left side
        for i in range(3):       # missionaries
            for j in range(3):   # cannibals
                if 1 <= i + j <= 2:   # boat can carry 1 or 2
                    new_state = (m1 - i, c1 - j, 0, m2 + i, c2 + j, 0)
                    if is_valid_state(new_state):
                        successors.append(new_state)
    else:
        # Boat is on the right side
        for i in range(3):
            for j in range(3):
                if 1 <= i + j <= 2:
                    new_state = (m1 + i, c1 + j, 1, m2 - i, c2 - j, 1)
                    if is_valid_state(new_state):
                        successors.append(new_state)

    return successors


def breadth_first_search() -> List[Tuple[int, int, int, int, int, int]]:
    """
    Solve the Missionaries and Cannibals problem using BFS.
    """
```

Document was last saved: Just now

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\Missionaries Cannibal
problem.py
Solution Path:
(3, 3, 1, 0, 0, 1)
(3, 1, 0, 0, 2, 0)
(3, 2, 1, 0, 1, 1)
(3, 0, 0, 0, 3, 0)
(3, 1, 1, 0, 2, 1)
(1, 1, 0, 2, 2, 0)
(2, 2, 1, 1, 1, 1)
(0, 2, 0, 3, 1, 0)
(0, 3, 1, 3, 0, 1)
(0, 1, 0, 3, 2, 0)
(0, 2, 1, 3, 1, 1)
(0, 0, 0, 3, 3, 0)
>>>
```

15. Write the python program for Vacuum Cleaner problem

vaccon cleaner.py - C:\Users\logan\AppData\Local\Programs\Python\Python313\vaccon cleaner.py (3.13.7)

```python
import random

def initialize_room(rows=4, cols=4):
    return [[1 for _ in range(cols)] for _ in range(rows)]

def detect_dirt(room):
    detected = [[random.choice([0, 1]) for _ in row] for row in room]
    return detected

def clean_room(room, detected):
    cleaned = 0
    total_dirty = sum(cell for row in detected for cell in row)
    for i in range(len(room)):
        for j in range(len(room[0])):
            if detected[i][j] == 1:
                print(f"Vaccum in this location now, {i} {j}")
                room[i][j] = 0
                print(f"cleaned {i} {j}")
                cleaned += 1
    return cleaned, total_dirty

def print_room(room):
    for row in room:
        print(row)

def main():
    print("All the rooms are dirty")
    room = initialize_room()
    print_room(room)

    print("\nBefore cleaning the room I detect all of these random dirts")
    detected = detect_dirt(room)
    print_room(detected)

    print()
    cleaned, total_dirty = clean_room(room, detected)

    print("\nRoom is clean now, Thanks for using : A.SAFARJI CLEANER")
    print_room(room)

    performance = (cleaned / (len(room) * len(room[0]))) * 100
    print(f"performance= {performance:.2f} %")

if __name__ == "__main__":
    main()
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:\Users\logan\AppData\Local\Programs\Python\Python313\vaccon cleaner.py
All the rooms are dirty
[1, 1, 1, 1]
[1, 1, 1, 1]
[1, 1, 1, 1]
[1, 1, 1, 1]

Before cleaning the room I detect all of these random dirts
[0, 0, 0, 0]
[1, 1, 1, 0]
[0, 0, 1, 0]
[1, 0, 0, 1]

Vaccum in this location now, 1 0
cleaned 1 0
Vaccum in this location now, 1 1
cleaned 1 1
Vaccum in this location now, 1 2
cleaned 1 2
Vaccum in this location now, 2 2
cleaned 2 2
Vaccum in this location now, 3 0
cleaned 3 0
Vaccum in this location now, 3 3
cleaned 3 3

Room is clean now, Thanks for using : A.SAFARJI CLEANER
[1, 1, 1, 1]
[0, 0, 0, 1]
[1, 1, 0, 1]
[0, 1, 1, 0]
performance= 37.50 %
>>>
```

16. Write the python program to implement BFS.

bfs.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/bfs.py (3.13.7)

```python
from collections import deque

# Define the graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Define the BFS function
def bfs(graph, start):
    visited = set()          # to keep track of visited nodes
    queue = deque([start])   # initialize queue with the starting node
    order = []               # to record traversal order

    while queue:
        node = queue.popleft()   # dequeue front element
        if node not in visited:
            visited.add(node)    # mark as visited
            order.append(node)   # record traversal
            # enqueue all unvisited neighbors
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)
    return order

# Call BFS starting from 'A'
print(bfs(graph, 'A'))
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>> ==== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/bfs.py ====
['A', 'B', 'C', 'D', 'E', 'F']
>>>
```

17. Write the python program to implement DFS.

dfs.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/dfs.py (3.13.7)

```python
# Define the graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Define the DFS function
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)  # mark the node as visited
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)  # recursively visit unvisited neighbors
    return visited

# Call the DFS function with starting node 'A'
print(dfs(graph, 'A'))
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>> ==== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/dfs.py ====
{'C', 'A', 'B', 'E', 'D', 'F'}
>>>
```

18. Write the python to implement Travelling Salesman Problem

salesman.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/salesman.py (3.13.7)

```python
import itertools
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)

start = 0

cities = list(range(n))
min_path = None
min_cost = float("inf")

for perm in itertools.permutations(cities[1:]):
    path = [start] + list(perm) + [start]     cost = 0
    for i in range(len(path) - 1):
        cost += graph[path[i]][path[i+1]]
    if cost < min_cost:
        min_cost = cost
        min_path = path

print("Optimal Path:", min_path)
print("Minimum Cost:", min_cost)
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/salesman.py =
Optimal Path: [0, 1, 3, 2, 0]
Minimum Cost: 80
>>>
```

19. Write the python program to implement A* algorithm

algorithm.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/algorithm.py (3.13.7)

```python
import heapq

def a_star(graph, start, goal, heuristic):
    open_list = [(heuristic[start], start, [start])]
    g_scores = {start: 0}
    visited = set()

    while open_list:
        f_score, current, path = heapq.heappop(open_list)

        if current == goal:
            return path, g_scores[current]

        if current in visited:
            continue
        visited.add(current)

        for neighbor, cost in graph.get(current, []):
            tentative_g = g_scores[current] + cost
            if tentative_g < g_scores.get(neighbor, float("inf")):
                g_scores[neighbor] = tentative_g
                f_neighbor = tentative_g + heuristic.get(neighbor, float("inf"))
                heapq.heappush(open_list, (f_neighbor, neighbor, path + [neighbor]))

    return None, float("inf")

# Test data
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 2)],
    'E': [('G', 1)],
    'F': [('G', 5)],
    'G': []
}

heuristic = {
    'A': 7, 'B': 6, 'C': 4, 'D': 2, 'E': 1, 'F': 3, 'G': 0
}

if __name__ == "__main__":
    path, cost = a_star(graph, 'A', 'G', heuristic)
    print("Path found:", path)
    print("Total cost:", cost)
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/a algorithm =
>>>
= RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/algorithm.py =
Path found: ['A', 'B', 'D', 'G']
Total cost: 4
>>>
```

20. Write the python program for Map Coloring to implement CSP.

mapcolouring.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/mapcolouring.py (3.13.7)

```python
        for constraint in self.constraints:
            if var in constraint and len(constraint) == 2:
                other_var = constraint[0] if constraint[1] == var else constraint[1]
                if other_var in assignments and assignments[other_var] == value:
                    conflicts += 1
        return conflicts

    def is_consistent(self, assignments):
        for constraint in self.constraints:
            if all(var in assignments for var in constraint):
                if not self.satisfies_constraint(assignments, constraint):
                    return False
        return True

    def satisfies_constraint(self, assignments, constraint):
        if len(constraint) == 2:
            val1, val2 = assignments[constraint[0]], assignments[constraint[1]]
            return val1 != val2

        else:
            return True

def map_coloring():
    variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
    domains = {
        'WA': ['r', 'g', 'b'],
        'NT': ['r', 'g', 'b'],
        'SA': ['r', 'g', 'b'],
        'Q': ['r', 'g', 'b'],
        'NSW': ['r', 'g', 'b'],
        'V': ['r', 'g', 'b'],
        'T': ['r', 'g', 'b']
    }
    constraints = [
        ('WA', 'NT'),
        ('WA', 'SA'),
        ('NT', 'SA'),
        ('NT', 'Q'),
        ('SA', 'Q'),
        ('SA', 'NSW'),
        ('SA', 'V'),
        ('Q', 'NSW'),
        ('NSW', 'V')
    ]
    csp = CSP(variables, domains, constraints)
    result = csp.solve()
    if result is not None:
        print(result)
    else:
        print("No solution found")
map_coloring()
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/mapcolouring.py
{'WA': 'r', 'NT': 'g', 'SA': 'b', 'Q': 'r', 'NSW': 'g', 'V': 'r', 'T': 'r'}
>>>
= RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/mapcolouring.py
{'WA': 'r', 'NT': 'g', 'SA': 'b', 'Q': 'r', 'NSW': 'g', 'V': 'r', 'T': 'r'}
>>>
```

21. Write the python program for Tic Tac Toe game

tic tac toe.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/tic tac toe.py (3.13.7)

```python
    # Check diagonals
    if all(board[i][i] == player for i in range(3)) or \
       all(board[i][2 - i] == player for i in range(3)):
        return True
    return False

def is_full(board):
    return all(cell != " " for row in board for cell in row)

def tic_tac_toe():
    board = [[" " for _ in range(3)] for _ in range(3)]
    players = ["X", "O"]
    turn = 0

    print("Welcome to Tic Tac Toe!")
    print_board(board)

    while True:
        player = players[turn % 2]
        print(f"Player {player}'s turn")

        try:
            row = int(input("Enter row (0,1,2): "))
            col = int(input("Enter col (0,1,2): "))
        except ValueError:
            print("Invalid input. Please enter numbers 0, 1, or 2.")
            continue

        if row not in range(3) or col not in range(3):
            print("Invalid position. Try again.")
            continue

        if board[row][col] != " ":
            print("Cell already taken. Try again.")
            continue

        board[row][col] = player
        print_board(board)

        if check_winner(board, player):
            print(f"Player {player} wins!")
            break

        if is_full(board):
            print("It's a draw!")
            break

        turn += 1

if __name__ == "__main__":
    tic_tac_toe()
```

IDLE Shell 3.13.7

```
Enter col (0,1,2): 0


  |   | O
-----
X |   |
-----
  | X |
-----


Player O's turn
Enter row (0,1,2): 2
Enter col (0,1,2): 0


  |   | O
-----
X |   |
-----
O | X |
-----


Player X's turn
Enter row (0,1,2): 2
Enter col (0,1,2): 2


  |   | O
-----
X |   |
-----
O | X | X
-----


Player O's turn
Enter row (0,1,2): 1
Enter col (0,1,2): 1


  |   | O
-----
X | O |
-----
O | X | X
-----


Player O wins!
>>>
```

22. Write the python program to implement Minimax algorithm for gaming

minmax.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/minmax.py (3.13.7)

```python
# Generic Minimax Algorithm Implementation

import math

# Minimax function
def minimax(depth, node_index, is_max, scores, height):
    # Base case: leaf node reached
    if depth == height:
        return scores[node_index]

    if is_max:
        return max(
            minimax(depth + 1, node_index * 2, False, scores, height),
            minimax(depth + 1, node_index * 2 + 1, False, scores, height)
        )
    else:
        return min(
            minimax(depth + 1, node_index * 2, True, scores, height),
            minimax(depth + 1, node_index * 2 + 1, True, scores, height)
        )

# Example usage
if __name__ == "__main__":
    # Example game tree (leaf node scores)
    scores = [3, 5, 6, 9, 1, 2, 0, -1]
    height = int(math.log2(len(scores)))

    print("Scores at leaf nodes:", scores)
    optimal_value = minimax(0, 0, True, scores, height)
    print("The optimal value is:", optimal_value)
```

IDLE Shell 3.13.7

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/minmax.py ==
Scores at leaf nodes: [3, 5, 6, 9, 1, 2, 0, -1]
The optimal value is: 5
>>>
```

23. Write the python program to implement Apha & Beta pruning algorithm for gaming

```python
# Alpha-Beta Pruning implementation in Python
# Example: Tic Tac Toe (can be adapted to other games)

import math

# Function to evaluate the board (example for Tic Tac Toe)
def evaluate(board):
    # Check rows
    for row in board:
        if row.count("X") == 3:
            return 10
        elif row.count("O") == 3:
            return -10

    # Check columns
    for col in range(3):
        if all(board[row][col] == "X" for row in range(3)):
            return 10
        elif all(board[row][col] == "O" for row in range(3)):
            return -10

    # Check diagonals
    if all(board[i][i] == "X" for i in range(3)):
        return 10
    elif all(board[i][i] == "O" for i in range(3)):
        return -10

    if all(board[i][2 - i] == "X" for i in range(3)):
        return 10
    elif all(board[i][2 - i] == "O" for i in range(3)):
        return -10

    return 0  # No winner yet
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/aplha beta.py
Current Board:
['X', 'O', 'X']
['O', 'O', ' ']
[' ', ' ', ' ']

The best move for X is at position: (1, 2)
>>>
```

24. Write the python program to implement Decision Tree

decision tree.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/decision tree.py (3.13.7)

File Edit Format Run Options Window Help

```python
# Decision Tree Classifier using scikit-learn
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text

# Load dataset (Iris flower dataset)
iris = load_iris()
X, y = iris.data, iris.target

# Create and train Decision Tree
clf = DecisionTreeClassifier(criterion="entropy", max_depth=3)
clf.fit(X, y)

# Display rules
tree_rules = export_text(clf, feature_names=iris.feature_names)
print(tree_rules)

# Predict example
sample = [[5.1, 3.5, 1.4, 0.2]]  # Sepal length, width, petal length, width
prediction = clf.predict(sample)
print("Predicted class:", iris.target_names[prediction][0])
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/decision tree.py
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2
|   |--- petal width (cm) >  1.75
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- class: 2
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: 2

Predicted class: setosa
>>>
```

25. Write the python program to implement Feed forward neural Network

FFNN.py - C:/Users/logan/AppData/Local/Programs/Python/Python313/FFNN.py (3.13.7)

File Edit Format Run Options Window Help

```python
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

y = np.array([[0],
              [1],
              [1],
              [0]])


np.random.seed(42)
input_layer_neurons = X.shape[1]
hidden_layer_neurons = 2
output_neurons = 1

W_hidden = np.random.uniform(size=(input_layer_neurons, hidden_layer_neurons))
b_hidden = np.random.uniform(size=(1, hidden_layer_neurons))
W_output = np.random.uniform(size=(hidden_layer_neurons, output_neurons))
b_output = np.random.uniform(size=(1, output_neurons))

lr = 0.1
epochs = 10000

for epoch in range(epochs):

    hidden_input = np.dot(X, W_hidden) + b_hidden
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W_output) + b_output
    final_output = sigmoid(final_input)

    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)

    error_hidden = d_output.dot(W_output.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    W_output += hidden_output.T.dot(d_output) * lr
    b_output += np.sum(d_output, axis=0, keepdims=True) * lr
    W_hidden += X.T.dot(d_hidden) * lr
    b_hidden += np.sum(d_hidden, axis=0, keepdims=True) * lr

print("Final Output after Training:")
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/logan/AppData/Local/Programs/Python/Python313/FFNN.py ===
Final Output after Training:
[[0.05322146]
 [0.95171535]
 [0.95160449]
 [0.05175396]]
>>>
```